

Reinforcement Learning Basics

23.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of **Machine Learning where an agent learns by interacting with an environment and receiving rewards or penalties.**

Instead of learning from labeled data like supervised learning, the model learns by **trial and error.**

The goal of the agent is to **maximize the total reward over time.**

23.2 Types of Machine Learning (Quick Revision)

There are mainly three types of Machine Learning:

Supervised Learning

Model learns from **labeled data.**

Example:

Input	Output
-------	--------

Email Text	Spam / Not Spam
------------	-----------------

The model learns the relationship between input and correct output.

Unsupervised Learning

Model learns **patterns in data without labels.**

Example:

- Customer segmentation
 - Clustering similar products
-

Reinforcement Learning

Model learns by **interacting with the environment and receiving rewards or penalties.**

Example:

A robot learns how to walk.

If it falls → penalty

If it walks properly → reward

23.3 Real-Life Example of Reinforcement Learning

Imagine you are training a **dog**.

Command:

Sit

Possible outcomes:

Dog sits → give a treat → reward

Dog ignores → no treat → penalty

Over time, the dog learns that **sitting gives rewards**.

This is exactly how Reinforcement Learning works.

23.4 Core Components of Reinforcement Learning

Reinforcement Learning consists of four important components:

1 Agent

2 Environment

3 Action

4 Reward

Agent

The **agent is the learner or decision-maker**.

Examples:

- A robot
- A self-driving car
- A game-playing AI
- A recommendation system

The agent's goal is to **take actions that maximize rewards**.

Environment

The **environment is everything the agent interacts with.**

Examples:

Agent	Environment
Chess AI	Chess board
Self-driving car	Road and traffic
Game AI	Video game world

The environment provides **feedback** to the agent.

Action

Actions are the **choices the agent can make.**

Example in a game:

Situation	Possible Actions
Character at crossroads	Move left, right, forward

The agent chooses an action based on what it has learned.

Reward

A reward is **feedback from the environment.**

Positive reward → good action

Negative reward → bad action

Example:

Action	Reward
Reach goal	+10
Hit obstacle	-5

The agent tries to **maximize total reward.**

23.5 RL Interaction Cycle

The process repeats continuously.

Step-by-step process:

- 1 Agent observes environment state
- 2 Agent selects an action
- 3 Environment changes state
- 4 Agent receives reward
- 5 Agent updates its learning

Cycle repeats many times.

Diagram Concept

Agent → Action → Environment

Environment → Reward + New State → Agent

23.6 State in Reinforcement Learning

A **state** represents the current situation of the environment.

Example in a maze:

State = current position of the player.

Example in chess:

State = current board configuration.

The agent must **choose the best action for each state**.

23.7 Policy

A **policy** is the strategy the agent follows.

It defines:

Which action to take in each state

Example:

State	Action
-------	--------

Enemy near	Attack
------------	--------

State	Action
-------	--------

Low health	Run
------------	-----

The policy improves as the agent learns.

23.8 Example: AI Learning to Play a Game

Suppose AI is learning a **simple maze game**.

Goal: reach the treasure.

Possible rewards:

Action	Reward
Move closer to treasure	+1
Reach treasure	+10
Hit wall	-2

The agent tries many paths and learns which path gives the **maximum reward**.

23.9 Exploration vs Exploitation

This is a very important concept in RL.

Exploration

Trying **new actions to discover better rewards**.

Example:

Trying a new path in the maze.

Exploitation

Using **known actions that already give good rewards**.

Example:

Following the best known path to treasure.

A good RL agent balances both.

Too much exploration → slow learning
Too much exploitation → may miss better solutions

23.10 Q-Learning (Basic Concept)

Q-Learning is one of the **most popular Reinforcement Learning algorithms**.

It helps the agent learn:

How good an action is in a particular state

The algorithm uses a **Q-Table**.

23.11 What is a Q-Table?

A Q-table stores **Q-values**, which represent the expected reward.

Example table:

State Left Right Up Down

S1 0 1.2 0.5 -0.3

S2 2.1 -1 0.7 0

The highest value tells the agent **which action is best**.

23.12 Q-Learning Update Formula

The Q-value updates using the formula:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Explanation of terms:

Symbol Meaning

s current state

a action

R reward

s' next state

α learning rate

Symbol Meaning

γ discount factor

Learning Rate (α)

Controls **how fast the agent learns**.

Example:

$\alpha = 0.1 \rightarrow$ slow learning

$\alpha = 0.9 \rightarrow$ fast learning

Discount Factor (γ)

Controls how much the agent values **future rewards**.

Example:

$\gamma = 0 \rightarrow$ only immediate reward matters

$\gamma = 0.9 \rightarrow$ future rewards also important

23.13 Simple Grid World Example

Imagine a grid:

S - -

- X -

- - G

S = Start

X = obstacle

G = Goal

Rewards:

Goal = +10

Obstacle = -5

Normal move = -1

The agent learns the **best path to reach the goal**.

Python Example (Simple Q-Learning Demonstration)

This is a **very simplified example** to help students understand the concept.

```
# Import numpy library for numerical calculations
```

```
import numpy as np
```

```
# Number of states in our environment
```

```
states = 5
```

```
# Number of possible actions
```

```
actions = 2
```

```
# Initialize Q-table with zeros
```

```
Q = np.zeros((states, actions))
```

```
# Learning rate
```

```
alpha = 0.1
```

```
# Discount factor
```

```
gamma = 0.9
```

```
# Total training episodes
```

```
episodes = 1000
```

```
# Loop through each training episode
```

```
for episode in range(episodes):
```

```
    # Start from state 0
```

```
    state = 0
```

```
    # Continue until goal state is reached
```

```
    while state != 4:
```

```
        # Choose a random action (exploration)
```

```
        action = np.random.randint(actions)
```

```
        # Define next state based on action
```

```
        if action == 0:
```

```
            next_state = max(0, state - 1)
```

```
        else:
```

```
            next_state = min(states - 1, state + 1)
```



```
# Reward system
if next_state == 4:
    reward = 10
else:
    reward = -1

# Update Q-value using Q-learning formula
Q[state, action] = Q[state, action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)

# Move to the next state
state = next_state

# Print the final Q-table
print("Learned Q-table:")
print(Q)
```

Code Explanation

Import Library

```
import numpy as np
```

Used for numerical operations and matrix handling.

Q-table Initialization

```
Q = np.zeros((states, actions))
```

Creates a table of **states × actions** initialized with zeros.

Learning Rate

```
alpha = 0.1
```

Controls how much new information affects learning.

Discount Factor

```
gamma = 0.9
```

Controls importance of **future rewards**.

Training Episodes

episodes = 1000

The agent repeats learning many times.

Random Action Selection

`action = np.random.randint(actions)`

Chooses random action for exploration.

Q-Value Update

$Q[\text{state}, \text{action}] = Q[\text{state}, \text{action}] + \alpha * (...)$

Updates the Q-value using the learning formula.

23.14 Real-World Applications

Game AI

AI systems that play games like:

- Chess
 - Go
 - Atari games
-

Robotics

Robots learn to:

- Walk
 - Pick objects
 - Navigate spaces
-

Self-Driving Cars

RL helps vehicles learn:

- Safe driving
 - Lane navigation
 - Traffic decision making
-

Recommendation Systems

Platforms like streaming services use RL to improve recommendations.

23.15 Famous Reinforcement Learning Achievements

AI has achieved major milestones:

- AI beating world champions in board games
 - Robots learning complex movements
 - Autonomous vehicles learning driving strategies
-

23.16 Limitations of Reinforcement Learning

Challenges include:

Training requires **huge computation**

Learning can take **millions of attempts**

Reward design must be **very carefully defined**

Practice Questions for Students

Question 1

What is Reinforcement Learning?

Question 2

Explain the following terms:

- Agent
- Environment

- Reward
-

Question 3

What is the difference between **Exploration and Exploitation**?

Question 4

What is a **Q-table**?

Coding Exercise

Modify the Python program so that:

- Goal reward = 20
- Penalty = -2

Observe how the Q-table changes.